

Transformers desde (casi) Cero - Clasificador de Reseñas en Español

<https://colab.research.google.com/github/cam2149/icesi-nlp/blob/Entrega3/Sesion3/Entrega3.ipynb>

Este notebook implementa un clasificador de reseñas en español utilizando la arquitectura Transformer construida desde cero. El proyecto demuestra cómo funciona internamente un modelo Transformer mediante la implementación manual de sus componentes principales.

Descripción del Proyecto

El notebook desarrolla un modelo de clasificación de sentimientos para reseñas de productos de Amazon en español, implementando los componentes clave de la arquitectura Transformer:

- **Multi-Head Self-Attention:** Mecanismo de atención que permite al modelo enfocarse en diferentes partes de la secuencia de entrada
- **Positional Encoding:** Codificación posicional para que el modelo entienda el orden de las palabras
- **Transformer Blocks:** Bloques encoder con conexiones residuales y normalización de capas
- **Clasificador:** Capa final para clasificación de reseñas (1-5 estrellas)

Objetivo

Clasificar reseñas de productos en español según su puntuación (1 a 5 estrellas) utilizando una implementación personalizada de la arquitectura Transformer.

Dataset

- **Fuente:** [Amazon Reviews Multi \(Español\)](#)
- **Idioma:** Español
- **Clases:** 5 (correspondientes a 1-5 estrellas)
- **Divisiones:** Train (200K), Validation (5K), Test (5K)
- **Campos:** review_title, review_body, review_id, stars

Componentes Implementados

1. Preprocesamiento de Datos

- Limpieza y normalización de texto
- Tokenización usando BERT español pre-entrenado
- Creación de datasets personalizados con PyTorch

2. Arquitectura del Modelo

- **TokenAndPosEmbedding:** Embeddings de tokens + codificación posicional
- **MultiHeadAttention:** Implementación completa de atención multi-cabeza
- **TransformerBlock:** Bloque encoder con feed-forward y conexiones residuales
- **SpanishNewsClassifier:** Modelo completo con PyTorch Lightning

3. Entrenamiento

- Optimizador AdamW con weight decay
- Early stopping para prevenir sobreajuste
- Métricas de accuracy para train/val/test
- Logging con TensorBoard

Requisitos

```
torch>=2.0.0
transformers>=4.20.0
datasets>=2.0.0
pytorch-lightning>=2.0.0
torchmetrics>=0.11.0
matplotlib>=3.5.0
seaborn>=0.11.0
pandas>=1.3.0
numpy>=1.21.0
tqdm>=4.62.0
optuna>=3.0.0
scikit-learn>=1.0.0
```

Cómo Usar

1. Instalación

El notebook instala automáticamente todas las dependencias necesarias.

2. Ejecución

```
# Si usas Jupyter Notebook
jupyter notebook Entrega3.ipynb

# Si usas Google Colab
# Simplemente abre el enlace del badge de Colab arriba
```

3. Configuración

Los principales parámetros configurables están al inicio:

```
max_len = 81          # Longitud máxima de secuencia (percentil 95)
emb_dim = 256          # Dimensión de embeddings
num_heads = 8          # Número de cabezas de atención
batch_size = 12        # Tamaño de lote
vocab_size = 50000     # Tamaño del vocabulario
```

Resultados Esperados

El modelo debería lograr:

- **Accuracy de entrenamiento:** ~85-90%
- **Accuracy de validación:** ~80-85%
- **Accuracy de prueba:** ~80-85%

Estructura del Código

```
|— 1. Configuración e Instalación
|— 2. Preprocesamiento de Datos
|   |— 2.1 Carga del Dataset
|   |— 2.2 Limpieza y Tokenización
|— 3. Implementación de Componentes
|   |— 3.1 Positional Encoding
|   |— 3.2 Multi-Head Attention
```

	└── 3.3 Transformer Block
	└── 4. Modelo Completo
	└── 4.1 Definición del Clasificador
	└── 4.2 Entrenamiento
	└── 5. Evaluación y Predicciones

🔍 Características Técnicas

- **Tokenizador:** BERT español pre-entrenado (dccuchile/bert-base-spanish-wwm-cased)
- **Vocabulario:** 50,000 tokens
- **Secuencia máxima:** 81 tokens (percentil 95)
- **Arquitectura:** Encoder-only Transformer
- **Optimización:** AdamW con $lr=2e-5$
- **Regularización:** Dropout 0.2, Weight decay $1e-5$

📖 Referencias

- [Attention is All You Need](#) - Paper original de Transformers
- [Natural Language Processing with Transformers](#) - Libro de referencia
- [Tutorial PyTorch Lightning](#) - Tutorial de transformers

🏠 Propósito Educativo

Este notebook está diseñado con fines educativos para:

- Comprender la arquitectura Transformer internamente
- Implementar mecanismos de atención desde cero
- Aprender sobre procesamiento de lenguaje natural en español
- Practicar con PyTorch Lightning y herramientas modernas de ML

📄 Licencia

Este proyecto es de código abierto y está disponible bajo la licencia MIT.

Nota: Este notebook fue desarrollado como parte del curso de NLP en ICESI. Para ejecutar en Google Colab, usa el badge de arriba o descarga el notebook localmente.

*
**

1. Entrega3.ipynb